
The Effect of Point Cloud Density on the performance of 3D Object Detection Model

Kim Junseok, Wu Xiaolong, Yao Junguang
the Chinese University of Hong Kong, ShenZhen
{224040305, 224010119, 224010136}@link.cuhk.edu.cn

Abstract

1 3D object detection, a critical technology in autonomous driving systems, fre-
2 quently encounters inefficiencies caused by dense point clouds, which increase
3 computational latency and memory consumption. This project investigates how
4 adjusting point cloud density through downsampling (random sampling, voxel grid
5 downsampling) and upsampling (Gaussian jittering, voxel interpolation) affects
6 the performance of 3D object detection models, focusing on balancing detection
7 accuracy (mAP) and efficiency. Using the PointPillars model pretrained on the
8 NuScenes dataset and testing on two samples (scene-061 and scene-103) from
9 NuScenes v1.0-mini, the study finds that voxel grid downsampling with a grid
10 size of 0.2 achieves a higher mAP (0.2397) than the original full-density input
11 (0.2342), while random downsampling degrades performance across all ratios. For
12 upsampling, both methods fail to match the original performance, but Gaussian
13 jittering shows better recovery. These results provide insights for optimizing point
14 cloud processing in real-time 3D perception applications, enabling more efficient
15 data usage without significant accuracy loss.

16 1 Introduction

17 3D object detection serves as the cornerstone of environmental perception in autonomous driving,
18 enabling vehicles to identify, localize, and classify surrounding objects such as cars, pedestrians,
19 and cyclists [3]. With the advancement of LiDAR technology, dense point clouds have become
20 the primary data source for 3D detection due to their ability to capture precise spatial information
21 [2]. However, the high density of these point clouds poses substantial challenges: they increase
22 computational complexity, extend processing time, and demand large memory resources, all of which
23 hinder the real-time performance required for safety-critical applications [6].

24 To address these issues, researchers have explored point cloud density adjustment techniques, pri-
25 marily downsampling and upsampling. Downsampling aims to reduce data volume while preserving
26 critical features. Common methods include random sampling, which selects points arbitrarily, and
27 voxel grid downsampling, which groups points into voxels and retains representative ones [7]. Up-
28 sampling, by contrast, seeks to recover lost information from sparse point clouds through methods
29 like Gaussian jittering (adding noise to generate new points) and voxel interpolation (estimating
30 missing points via neighboring values) [12].

31 Despite their potential, existing studies often lack systematic comparisons of these techniques. While
32 some focus on downsampling for efficiency [4], others explore upsampling for accuracy recovery
33 [10], but few evaluate their combined effects across diverse scenarios. Additionally, the optimal
34 parameters for these methods (e.g., voxel size in downsampling) and their impact on different object
35 categories remain underexplored.

This project fills this gap by comprehensively evaluating downsampling and upsampling methods on 3D object detection performance. Using the PointPillars model—pretrained on the NuScenes dataset—and two representative samples (a dense urban scene and a sparse environment), we quantify how density adjustments influence mAP and efficiency. The goal is to identify optimal strategies that balance accuracy and computational demands, providing practical guidelines for real-world applications like autonomous navigation.

1.1 Related Work

this section synthesizing key research on point cloud density adjustment for 3D object detection:

Adaptive Downsampling Methods: these methods have evolved to address information loss in point cloud reduction. Early techniques like random sampling and farthest point sampling prioritized computational efficiency but risked discarding critical geometric features[6]. Structured approaches such as voxel grid downsampling improved spatial coherence by aggregating points within fixed volumetric units, yet optimal voxel size selection remained heuristic [4]. Recent learning-based methods like Adaptive Hierarchical Downsampling introduced Critical Points Layers to retain task-relevant points via feature-maximization, achieving SOTA classification on ModelNet40 while maintaining permutation invariance[5]. Similarly, AVS-Net dynamically adjusted voxel sizes using point-density priors, balancing geometric preservation and efficiency on ScanNet benchmarks[9].

Task-Oriented Sampling Strategies: these strategies focus on foreground preservation. IA-SSD proposed instance-aware downsampling, combining class-aware and centroid-aware sampling to prioritize foreground points near object centers. This reduced computation while improving pedestrian/cyclist detection in sparse regions[11]. For multi-class imbalance, CBGS grouped geometrically similar categories (e.g., car/ truck/construction vehicle) into balanced clusters. Its density-based sampling oversampled rare classes (e.g., bicycles), boosting nuScenes detection mAP by 12% over PointPillars [13].

Multimodal Fusion techniques: these techniques address density-induced noise. VirConv tackled virtual-point densification from RGB-guided depth completion. Its Stochastic Voxel Discard dropped redundant nearby voxels (97% of points contributed <0.2% mAP gain), while Noise-Resistant Conv suppressed boundary artifacts via 2D-3D feature fusion, achieving KITTI SOTA [8]. For upsampling, MonoDGAE enhanced sparse monocular detections using depth-guided attention with bilateral filtering, refining spatial coherence in cluttered scenes [1].

1.2 Contribution

Kim Junseok is responsible for data pre-processing, part of presentation slides. Yao Junguang is responsible for part of coding, part of report. WU Xiaolong is responsible for part of coding, part of presentation slides and part of report. Our core contribution is about exploring the relationship between point clouds density and model performance, we also explored which data processing method yields the best trade-off between performance and density.

2 The Proposed Algorithm

In this section, we will describe our methods for point cloud data processing, including down sampling methods and up sampling methods. To verify the conclusion more fairly, we adopt different methods for both of the two data processing direction. For down sampling methods, we adopt 'random down sampling' and 'voxel grid down sampling'. For up sampling methods, we adopt 'Gaussian Jittering' and 'Voxel Interpolation' methods.

2.1 Random down sampling

For random down sampling, the mechanism of this method is just like its name, where we randomly sample part of the points of the original point clouds to get a sparser representation. The pseudo code of this algorithm is in algorithm 1.

Algorithm 1: Random down sampling

Data: original point clouds $P = (X_i, Y_i, Z_i)_{i=1}^N$, sampling ratio r

Result: sparse representation $P_S = (X_i, Y_i, Z_i)_{i=1}^{N_r}$

```
1  $N_r = r \cdot N$ ;  
2  $i = 1$ ;  
3  $P_s = \phi$ ;  
4 while  $i \leq N_r$  do  
5   randomly choose  $u \in P$ ;  
6    $P_S = P_S \cup u$ ;  
7 end
```

82 2.2 Voxel grid down Sampling

83 In voxel grid down sampling, it's unlike random down sampling, we sample a part of the points from
84 the original point clouds that satisfy a certain pattern. Specifically, we firstly define a voxel grid size
85 r , then we splits the total point clouds region into small grids with size r , and get the voxel grid
86 numbers per dimension (D_X, D_Y, D_Z) .

$$87 \quad D_X = (X_{max} - X_{min})/r \quad (1)$$

$$88 \quad D_Y = (Y_{max} - Y_{min})/r \quad (2)$$

$$89 \quad D_Z = (Z_{max} - Z_{min})/r \quad (3)$$

89 After that, we assign every 3d point with an index set (h_X, h_Y, h_Z) , which denotes which voxel
90 grid it belongs to. Further, We perform average operation of all the points inside every voxel grid to
91 get only one 3d point. Finally, we concatenate all of these average points together to get a sparser
92 representation from a voxel grid level. The pseudo code is in algorithm 2.

93 Despite the fact that this down sampling method is more computational complex than the random
94 down sampling, it may has a better down sampling performance than the random one: since it divides
95 the whole point cloud space into small grids and get only one point from every grid, so it may preserve
96 a more complete global spatial structure. Besides, the density of the down sampled point clouds
97 by this way is much more uniform everywhere than the random down sampling. for random down
98 sampling, although we sample data points following a uniform distribution, the global density of the
99 point clouds can still not be uniform everywhere, this may hinders the performance of the 3d point
100 clouds based DNN.

101 So we choose this down sampling method as a counterpart of the random down sampling, our
102 experimental results also proved the advantage of voxel grid down sampling over the random down
103 sampling.

104 2.3 Gaussian jittering up sampling

105 For up sampling method, we firstly use 'Gaussian Jittering' method. In this algorithm, we up sample
106 a dense point clouds from a sparse one. Let N denotes the number of the points in the original point
107 clouds, and M denotes the number of the points in the desired up sampled point clouds. We firstly
108 choose $M - N$ points from the original point clouds P randomly, we denote this point clouds as P_C ,
109 then we add gaussian noise to P_C . Finally, we concatenate the P and P_C together to get a denser
110 representation P_D , that is: $P_D = P \cup P_C$. The pseudo code is in algorithm 3.

111 2.4 Voxel interpolation up sampling

112 For voxel interpolation, the idea is just like voxel grid down sampling, where we generate new points
113 inside every voxel grid. Specifically, firstly we also divide the whole point clouds space into small
114 voxel grids. After that, we select two points inside every voxel grid and calculate the mean point
115 of these two points. Lastly we concatenate the mean points with the original points together to get
116 a denser representation. If the number is still smaller than the desired number, we just repeat the
117 interpolation until the condition is satisfied. The pseudo code is in algorithm 4.

Algorithm 2: Voxel grid down sampling

Data: original point clouds $P = (X_i, Y_i, Z_i)_{i=1}^N$, voxel grid size r

Result: sparse representation $P_S = (X_i, Y_i, Z_i)_{i=1}^{N_r}$

```
1  $D_X = (X_{max} - X_{min})/r$ ;  
2  $D_Y = (Y_{max} - Y_{min})/r$ ;  
3  $D_Z = (Z_{max} - Z_{min})/r$ ;  
4  $h_X = ((x_i)_{i=1}^N - X_{min})/r$ ;  
5  $h_Y = ((Y_i)_{i=1}^N - Y_{min})/r$ ;  
6  $h_Z = ((Z_i)_{i=1}^N - Z_{min})/r$ ;  
7  $P_s = \phi$ ;  
8 for  $i \leq h_X$  do  
9   for  $j \leq h_Y$  do  
10    for  $k \leq h_Z$  do  
11       $S = \{(X_j, Y_j, Z_j) | (h_X)_j = i, (h_Y)_j = j, (h_Z)_j = k\}$ ;  
12       $a = \text{mean}(S)$ ;  
13       $P_s = P_s \cup a$ ;  
14    end  
15  end  
16 end  
17
```

Algorithm 3: Gaussian Jittering

Data: original point clouds $P = (X_i, Y_i, Z_i)_{i=1}^N$, target point cloud number M

Result: dense representation $P_D = (X_i, Y_i, Z_i)_{i=1}^M$

```
1 randomly choose  $M - N$  points  $P_C$  from  $P$ ;  
2 for  $(x, y, z)$  in  $P_C$  do  
3    $x = x + N(0, 0.01)$ ;  
4    $y = y + N(0, 0.01)$ ;  
5    $z = z + N(0, 0.01)$ ;  
6 end  
7  $P_D = P \cup P_C$ 
```

Algorithm 4: Voxel interpolation

Data: original point clouds $P = (X_i, Y_i, Z_i)_{i=1}^N$, target point cloud number M

Result: dense representation $P_D = (X_i, Y_i, Z_i)_{i=1}^M$

```
1  $P_D = P_C$ ;  
2 while  $|P_D| \neq M$  do  
3   Divide  $P$  into voxel grids  $V = (v_i)_{i=1}^t$ ;  
4   for  $(v_i)$  in  $V$  do  
5     select  $(x1, y1, z1)$  and  $(x2, y2, z2) \in v_i$ ;  
6      $(x, y, z) = 0.5 * (x1, y1, z1) + 0.5 * (x2, y2, z2)$ ;  
7      $P_D = P_D \cup (x, y, z)$ ;  
8     if  $|P_D| = M$  then  
9       break  
10    end  
11  end  
12 end
```

3 Experiments

For experiments, we selected a 'PointPillars' which has been pretrained on Nuscenes dataset as our base model. We selected 2 samples of 'nuscenes-v1.0-mini' as our dataset, specifically, their information is in table 1. The down sampling ratios of random down sampling are set as: 25%, 50% and 100%(original), the voxel sizes for voxel grid are set as 0.4, 0.2 and 0.05. For up sampling, we use two setting, in the first setting, our objective is to restore the 25% randomly downsampled point clouds back to the original size, in the second setting, our objective is to restore the 50% randomly downsampled point clouds back to the original size. For gaussian jittering, we set the standard error of the gaussian noise as 0.01, and for voxel interpolation, we set the voxel grid size as 0.4. For evaluating metric, we adopt the mAP.

Table 1: Your first table.

	LIDAR_TOP FILE
sample 1	n015-2018-07-24-11-22-45+0800__LIDAR_TOP__1532402937198682.pcd.bin
sample 2	n008-2018-08-01-15-16-36-0400__LIDAR_TOP__1533151613398020.pcd.bin

The results for down sampling are in 1.

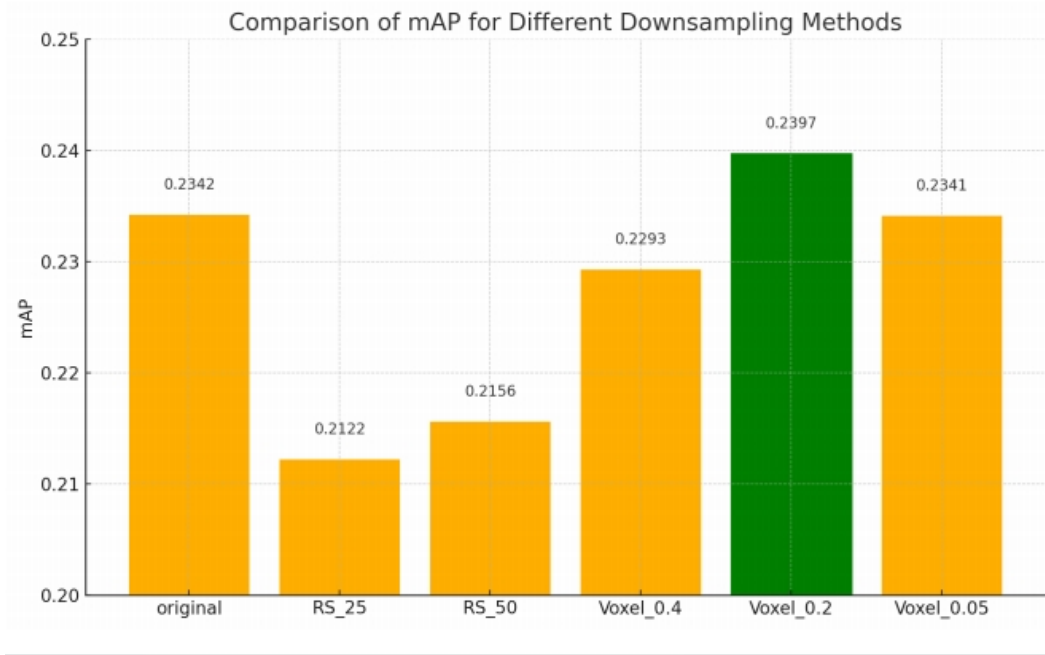


Figure 1: Numerical results for down sampling

The results for up sampling are in 2.

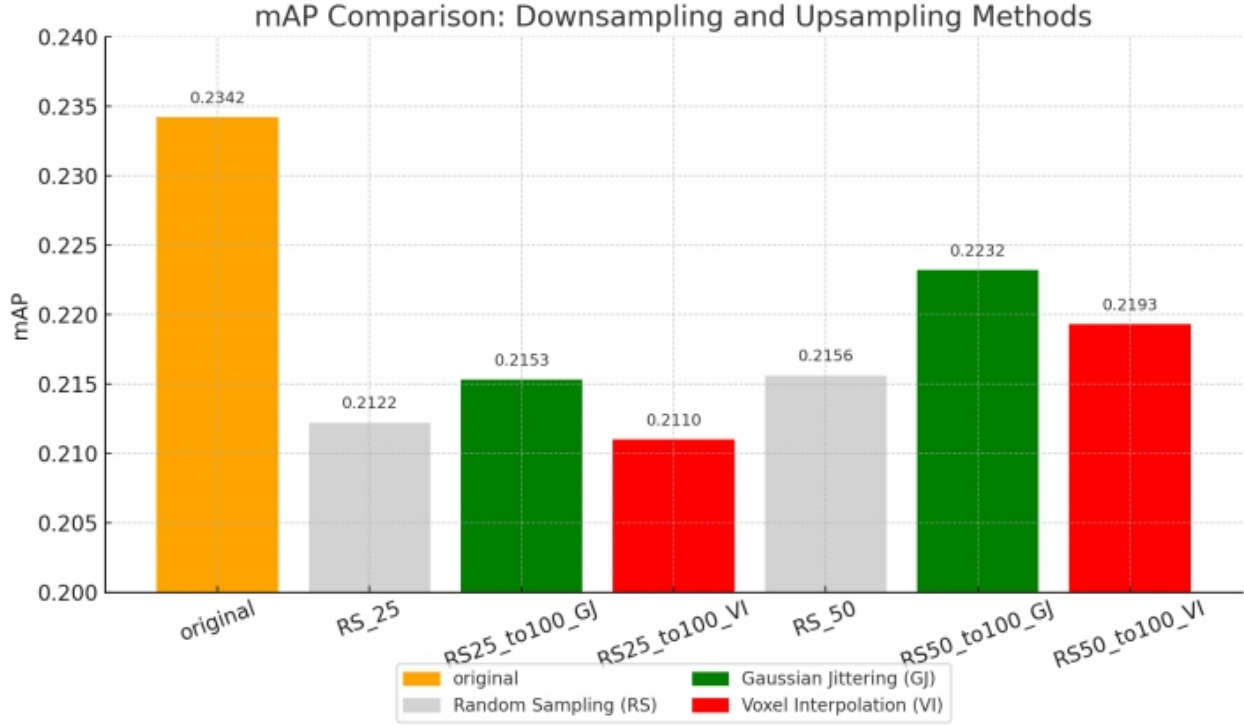


Figure 2: Numerical results for up sampling

4 Discussions

From figure 1, we can see that, all of the random down sampling method have performance degrade, and the sparser the point clouds, the worse the performance, this means that, the random down sampling method is not robust. On the contrary, for voxel grid downsampling method, we can see that, if the grid size is proper(e.g., 0.2), the performance can even be enhanced. For the reason behind this phenomenon, we think that, a proper down sampling method can act as a denoising method, which eliminates noise that may hinder the prediction, while preserving a relatively complete spatial structure.

For up sampling, we can see from figure 2 that, all of the up sampling methods are worse than the baseline, which means that, actually, up sampling is a challenge for 3d point clouds, unlike 2D computer vision task. But gaussian jittering may be a better trade-off than the voxel interpolation. The reason behind this phenomenon, we guess, is that, an improper up sampling method may add noise to the point clouds data, which blur the spatial structure and make the prediction much harder.

References

- [1] G. A. Bitwire, S. Kakuba, D. W. Cha, and D. S. Han. Monodgae: depth-guided attention and bilateral filtering for robust monocular 3d object detection. *Artificial Life and Robotics*, pages 1–12, 2025.
- [2] J. Cheng, Z. Xiang, T. Cao, and J. Liu. Robust vehicle detection using 3d lidar under complex urban environment. In *2014 IEEE international conference on robotics and automation (ICRA)*, pages 691–696. IEEE, 2014.
- [3] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.

- 153 [4] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom. Pointpillars: Fast encoders
154 for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer
155 vision and pattern recognition*, pages 12697–12705, 2019.
- 156 [5] E. Nezhadarya, E. Taghavi, R. Razani, B. Liu, and J. Luo. Adaptive hierarchical down-sampling
157 for point cloud classification. In *Proceedings of the IEEE/CVF conference on computer vision
158 and pattern recognition*, pages 12956–12964, 2020.
- 159 [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. Pointnet++: Deep hierarchical feature learning on
160 point sets in a metric space. *Advances in neural information processing systems*, 30, 2017.
- 161 [7] R. B. Rusu and S. Cousins. 3d is here: Point cloud library (pcl). In *2011 IEEE international
162 conference on robotics and automation*, pages 1–4. IEEE, 2011.
- 163 [8] H. Wu, C. Wen, S. Shi, X. Li, and C. Wang. Virtual sparse convolution for multimodal 3d
164 object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern
165 recognition*, pages 21653–21662, 2023.
- 166 [9] H. Yang, D. Liang, D. Zhang, X. Jiang, Z. Liu, Z. Zou, and Y. Zhu. Avs-net: Point sampling
167 with adaptive voxel size for 3d point cloud analysis. *arXiv preprint arXiv:2402.17521*, 5(7),
168 2024.
- 169 [10] L. Yu, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng. Pu-net: Point cloud upsampling network.
170 In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages
171 2790–2799, 2018.
- 172 [11] Y. Zhang, Q. Hu, G. Xu, Y. Ma, J. Wan, and Y. Guo. Not all points are equal: Learning highly
173 efficient point-based detectors for 3d lidar point clouds. In *Proceedings of the IEEE/CVF
174 conference on computer vision and pattern recognition*, pages 18953–18962, 2022.
- 175 [12] Y. Zhou and O. Tuzel. Voxelnet: End-to-end learning for point cloud based 3d object detection.
176 In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages
177 4490–4499, 2018.
- 178 [13] B. Zhu, Z. Jiang, X. Zhou, Z. Li, and G. Yu. Class-balanced grouping and sampling for point
179 cloud 3d object detection. *arXiv preprint arXiv:1908.09492*, 2019.

180 Acknowledgments

181 Thanks for Prof. LI, T.A. HUANG AND T.A. LI’s instruction and help.